Julia Miyamoto
ATLS 2270
Computer Foundations 2
Spring 2026

Final Project - Bar Hopping Mapper

For my final project I created a bar mapping system that plans the perfect bar crawl for you based on bars you input in your area. The project explores how graph data structures and shortest-path algorithms can be applied to real-world and in this case planning an efficient customizable bar crawl. The goal was to design a system that models bars as nodes in a graph and walking paths between them as weighted edges, allowing users to generate optimized crawl routes based on performances like distance, rating, cost, and happy hour availability.

At the core of the project is a graph data structure, implemented using an adjacency list. Each bar is represented as a node containing attributes such as name, neighborhood, rating, cover charge, closing time, and whether it offers happy hour. Connections between cars are stored as edges with weights representing walking/biking time in minutes. This structure allows efficient traversal and flexible modeling of real-world spatial relationships.

```
export class BarGraph {
   private adjacency: Map<string, Edge[]> = new Map();
   private bars: Map<string, Bar> = new Map();

   addBar(bar: Bar): void {
       this.bars.set(bar.name, bar);
       if (!this.adjacency.has(bar.name)) {
           this.adjacency.set(bar.name, []);
       }
   }
   public addEdge(from: string, to: string, distanceMinutes:number):void{
       this.adjacency.get(from)?.push({to, distance: distanceMinutes});

       this.adjacency.get(to)?.push({to: from, distance: distanceMinutes});
   }
   getBar(name: string): Bar | undefined {
       return this.bars.get(name);
   }
   getNeighbors(barName: string): Edge[] {
       return this.adjacency.get(barName) ?? [];
   }
   getAllBars(): Bar[] {
       return Array.from(this.bars.values());
   }
   hasBar(name: string): boolean {
```

```
        return this.bars.has(name);
    }
```

To compute optimal routes, I implemented a Dijkstra's algorithm, after consulting my comp sci friend for advice. This algorithm finds the shortest path between a starting bar and all the other bars in the graph. This algorithm is particularly useful because all edge weights are non-negative. The algorithm maintains a distance map and a previous-node map to reconstruct the shortest paths. While an array-based priority queue was used initially, I also implemented a min-heap class to demonstrate a more efficient approach and reducing complexity time.

```
dijkstra(start: string): DijkstraResults {
    const distance = new Map<string, number>();
    const previous = new Map<string, string | null>();
    const visited = new Set<string>();

    for (const name of this.bars.keys()) {
        distance.set(name, Infinity);
        previous.set(name, null);
    }
    distance.set(start, 0);

    const queue: string[] = [start];
    while (queue.length > 0) {
        queue.sort((a, b) => (distance.get(a) ?? Infinity) - (distance.get(b) ??
Infinity));
        const current = queue.shift()!;
        if (visited.has(current)) continue;
        visited.add(current);
        for (const edge of this.getNeighbors(current)) {
            if (visited.has(edge.to)) continue;
            const newDist = (distance.get(current) ?? Infinity) + edge.distance;
            if (newDist < (distance.get(edge.to) ?? Infinity)) {
                distance.set(edge.to, newDist);
                previous.set(edge.to, current);
                queue.push(edge.to);
            }
        }
    }
    return { distance, previous };
}
```

Building on this, I created a planCrawl function that simulates a bar crawl experience. Starting from a chosen bar, the function repeatedly selects the next closest valid bar based on user-defined constraints such as max. number of stops, min. rating, max. rating, max. cover charge, and weather to include happy hour only locations. It also tracks time progression, assuming you spend a fixed amount of time at each bar. The function prints a detailed itinerary including arrival times and walking distances.

```
planCrawl(
   start: string,
   options:{
       maxStops?: number;
       happyHourOnly?: boolean;
       maxCoverCharge?: number;
       minRating?: number;
       startTime?: number;
   }={}
): CrawlStop[]{
   const{
       maxStops = 5,
       happyHourOnly = false,
       maxCoverCharge = Infinity,
       minRating = 0,
       startTime = 20*60,
   } = options;
   const visited = new Set<string>();
   const crawl: CrawlStop[] = [];
   let current = start;
   let currentTime = startTime;

   const startBar = this.getBar(start);
   if (!startBar) return [];
   crawl.push({bar: startBar, arrivalTime: 0, distanceFromPrev: 0});
   visited.add(start);

   while (crawl.length < maxStops){
       const {distance, previous} = this.dijkstra(current);
       let nextBar: string | null = null;
       let bestDist = Infinity;

       for (const [name, dist] of distance.entries()){
           if (visited.has(name)) continue;
           const bar = this.getBar(name);
           if(!bar) continue;
```

```
            if (happyHourOnly && !bar.happyHour) continue;
            if (bar.coverCharge > maxCoverCharge) continue;
            if (bar.rating < minRating) continue;

            if (dist < bestDist){
                bestDist = dist;
                nextBar = name;
            }
        }

        if (!nextBar) break;

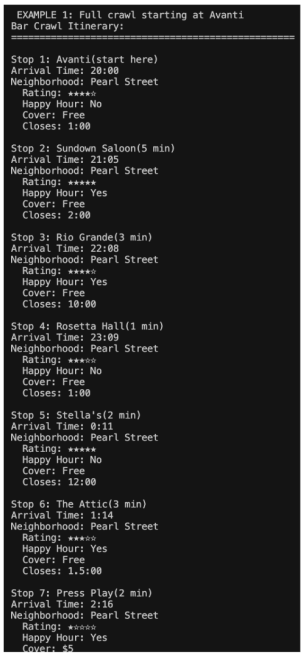
        const bar = this.getBar(nextBar)!;
        currentTime += bestDist + 60;
        crawl.push({
            bar,
            arrivalTime: currentTime - startTime,
            distanceFromPrev: bestDist,
        });
        visited.add(nextBar);
        current = nextBar;
    }
    return crawl;
}
}
```

One of the most interesting aspects of the project is how algorithmic decisions-making interacts with certain constraints. For example, the shortest path is not always the best choice if a bar does not meet user preferences. This required layering filtering logic on top of the graph transversal, effectively combining optimization with constraint satisfaction.

The most challenging part of this project was figuring out how to correctly implement Dijkstra's algorithm from scratch. While we learned about graphs, actually coding the algorithm required combining multiple course topics. In particular, I had to think about the difference between abstract and concrete data types, specifically how priority queues should behave versus how to implement it efficiently using a binary heap. Another difficulty was debugging the algorithm and ensuring correctness. Small mistakes, like not updating distance properly or revisiting nodes could break the results.

To test this system, I created two separate datasets based on real-world locations. One being bars in Boulder, Co and the other being bars in Los Angeles, specifically the South Bay. This allowed me to verify that the graph structure and algorithm worked across different layouts

and densities of locations. The Boulder dataset had many closely connected bars with short walking distances, which tested if the algorithm could quickly find efficient multi-stop crawls. The South Bay dataset, included longer distances between clusters of bars which tested whether the algorithm could still compute reasonable paths across larger gaps.



```
 EXAMPLE 1: Full crawl starting at Avanti
Bar Crawl Itinerary:
==================================================

Stop 1: Avanti(start here)
Arrival Time: 20:00
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: No
  Cover: Free
  Closes: 1:00

Stop 2: Sundown Saloon(5 min)
Arrival Time: 21:05
Neighborhood: Pearl Street
  Rating: ★★★★★
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

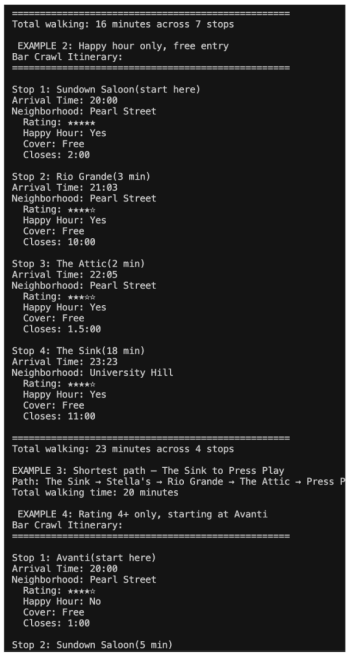
Stop 3: Rio Grande(3 min)
Arrival Time: 22:08
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 10:00

Stop 4: Rosetta Hall(1 min)
Arrival Time: 23:09
Neighborhood: Pearl Street
  Rating: ★★★½☆
  Happy Hour: No
  Cover: Free
  Closes: 1:00

Stop 5: Stella's(2 min)
Arrival Time: 0:11
Neighborhood: Pearl Street
  Rating: ★★★★★
  Happy Hour: No
  Cover: Free
  Closes: 12:00

Stop 6: The Attic(3 min)
Arrival Time: 1:14
Neighborhood: Pearl Street
  Rating: ★★★½☆
  Happy Hour: Yes
  Cover: Free
  Closes: 1.5:00

Stop 7: Press Play(2 min)
Arrival Time: 2:16
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: $5
```



```
==================================================
Total walking: 16 minutes across 7 stops

 EXAMPLE 2: Happy hour only, free entry
Bar Crawl Itinerary:
==================================================

Stop 1: Sundown Saloon(start here)
Arrival Time: 20:00
Neighborhood: Pearl Street
  Rating: ★★★★★
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

Stop 2: Rio Grande(3 min)
Arrival Time: 21:03
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 10:00

Stop 3: The Attic(2 min)
Arrival Time: 22:05
Neighborhood: Pearl Street
  Rating: ★★★½☆
  Happy Hour: Yes
  Cover: Free
  Closes: 1.5:00

Stop 4: The Sink(18 min)
Arrival Time: 23:23
Neighborhood: University Hill
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 11:00

==================================================
Total walking: 23 minutes across 4 stops

EXAMPLE 3: Shortest path — The Sink to Press Play
Path: The Sink → Stella's → Rio Grande → The Attic → Press P
Total walking time: 20 minutes

 EXAMPLE 4: Rating 4+ only, starting at Avanti
Bar Crawl Itinerary:
==================================================

Stop 1: Avanti(start here)
Arrival Time: 20:00
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: No
  Cover: Free
  Closes: 1:00

Stop 2: Sundown Saloon(5 min)
```



```
 EXAMPLE 4: Rating 4+ only, starting at Avanti
Bar Crawl Itinerary:
==================================================

Stop 1: Avanti(start here)
Arrival Time: 20:00
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: No
  Cover: Free
  Closes: 1:00

Stop 2: Sundown Saloon(5 min)
Arrival Time: 21:05
Neighborhood: Pearl Street
  Rating: ★★★★★
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

Stop 3: Rio Grande(3 min)
Arrival Time: 22:08
Neighborhood: Pearl Street
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 10:00

Stop 4: Stella's(1 min)
Arrival Time: 23:09
Neighborhood: Pearl Street
  Rating: ★★★★★
  Happy Hour: No
  Cover: Free
  Closes: 12:00

==================================================
Total walking: 9 minutes across 4 stops
```



```
EXAMPLE 1: Full crawl starting at Tony's on the Pier
Bar Crawl Itinerary:
==================================================

==================================================
Total walking: 0 minutes across 0 stops

EXAMPLE 2: Happy hour only, free entry
Bar Crawl Itinerary:
==================================================

Stop 1: Shellback Tavern(start here)
Arrival Time: 20:00
Neighborhood: Manhattan Beach
  Rating: ★★★★★
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

Stop 2: The Hangar Inn(3 min)
Arrival Time: 21:03
Neighborhood: Manhattan Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 12:00

Stop 3: Tower 12(15 min)
Arrival Time: 22:18
Neighborhood: Hermosa Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

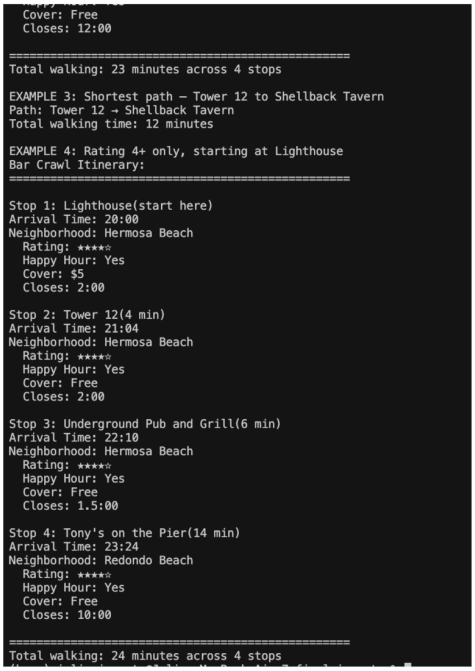
Stop 4: The Poop Deck(5 min)
Arrival Time: 23:23
Neighborhood: Hermosa Beach
  Rating: ★★★½☆
  Happy Hour: Yes
  Cover: Free
  Closes: 12:00

==================================================
Total walking: 23 minutes across 4 stops

EXAMPLE 3: Shortest path — Tower 12 to Shellback Tavern
Path: Tower 12 → Shellback Tavern
Total walking time: 12 minutes

EXAMPLE 4: Rating 4+ only, starting at Lighthouse
Bar Crawl Itinerary:
==================================================

Stop 1: Lighthouse(start here)
Arrival Time: 20:00
Neighborhood: Hermosa Beach
  Rating: ★★★★☆
  Happy Hour: Yes
```



```
  Cover: Free
  Closes: 12:00

==================================================
Total walking: 23 minutes across 4 stops

EXAMPLE 3: Shortest path — Tower 12 to Shellback Tavern
Path: Tower 12 → Shellback Tavern
Total walking time: 12 minutes

EXAMPLE 4: Rating 4+ only, starting at Lighthouse
Bar Crawl Itinerary:
==================================================

Stop 1: Lighthouse(start here)
Arrival Time: 20:00
Neighborhood: Hermosa Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: $5
  Closes: 2:00

Stop 2: Tower 12(4 min)
Arrival Time: 21:04
Neighborhood: Hermosa Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 2:00

Stop 3: Underground Pub and Grill(6 min)
Arrival Time: 22:10
Neighborhood: Hermosa Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 1.5:00

Stop 4: Tony's on the Pier(14 min)
Arrival Time: 23:24
Neighborhood: Redondo Beach
  Rating: ★★★★☆
  Happy Hour: Yes
  Cover: Free
  Closes: 10:00

==================================================
Total walking: 24 minutes across 4 stops
```

Overall, this project demonstrates how multiple core topics for the class including graphs, abstract data types, heaps and complexity, come together for one practical application. By testing with real bars that I have been to, I was able to confirm that the system is not only theoretically

sound but also useful and adaptable in realistic scenarios. I'm excited to actually try out one of the generated bar crawl itineraries this weekend to see how well it works. It'll be a fun way to test whether the algorithm's decisions feel natural while also enjoying a night out. I think I'm going to do this route:
Stop 1: Stella's(start here)
Arrival Time: 20:00
Neighborhood: Pearl Street
Rating: ★★★★★
Happy Hour: No
Cover: Free
Closes: 12:00

Stop 2: Rio Grande(1 min)
Arrival Time: 21:01
Neighborhood: Pearl Street
Rating: ★★★★☆
Happy Hour: Yes
Cover: Free
Closes: 10:00

Stop 3: Rosetta Hall(1 min)
Arrival Time: 22:02
Neighborhood: Pearl Street
Rating: ★★★☆☆
Happy Hour: No
Cover: Free
Closes: 1:00

Stop 4: The Attic(3 min)
Arrival Time: 23:05
Neighborhood: Pearl Street
Rating: ★★★☆☆
Happy Hour: Yes
Cover: Free
Closes: 1.5:00
(Hopefully it's fun)

Link to code: https://gist.github.com/juliamiyamoto/5c6a8d4c92c3531c31c284dbd55f5cab